Assignment No : 1

Name : Om A. Akiwate

PRN : 255209003

Date : 21-07-2026

Batch : BT4

Objectives:

• Set up and use a Python environment (Python with IDE or Jupyter Notebook) without any installation headaches.

• Use variables, lists, loops, conditionals, and functions, the minimum Python needed for ML.

• Represent data as vectors using NumPy and compute mean, variance, and standard deviation.

• Load a CSV with pandas, summarize it, and draw your first histogram and scatter plot.

```
print("Hello, Machine Learning!")
```
[1] ✓ 0.0s Python

```
Hello, Machine Learning!
```

Dataset :

```
names = ["Aarav","Diya","Ishaan","Meera","Kabir","Anaya",
"Rohan","Sara","Vivaan","Priya","Arjun","Nisha"]

test1 = [56,78,45,88,62,91,40,73,67,85,52,95]

test2 = [61,82,50,90,58,94,38,70,72,88,47,97]

attendance = [72,88,65,95,70,98,55,80,78,92,60,99]
```
[2] ✓ 0.0s Python

A1. Variables and printing :

```
topper = "Nisha"
marks= 95
print(topper , "Scored", marks , "in Test 1")
```
[3] ✓ 0.0s Python

```
Nisha Scored 95 in Test 1
```

```
name="Om"
mark= 99
print(name , "Expected to score" , mark, "marks")
```
[4] ✓ 0.0s Python

```
Om Expected to score 99 marks
```

A2. Lists and Indexing :

```
print(test1[0])
print(test1[-1])
print(len(test1))
```

[5] ✓ 0.0s Python

```
56
95
12
```

```
print(names[4] , test1[4])
```

[6] ✓ 0.0s Python

```
Kabir 62
```

A3. Loops :

```
for i in range(len(names)) :
    print(names[i] , test1[i])
```

[7] ✓ 0.0s Python

```
Aarav 56
Diya 78
Ishaan 45
Meera 88
Kabir 62
Anaya 91
Rohan 40
Sara 73
Vivaan 67
Priya 85
Arjun 52
Nisha 95
```

```
for i in range(len(names)) :
    improvement = test2[i] - test1[i]
    print("Names :", names[i] , "Test1 :", test1[i] ,"Test2 :", test2[i], "Improvement = " , improvement)
```

[8] ✓ 0.0s Python

```
Names : Aarav Test1 : 56 Test2 : 61 Improvement =  5
Names : Diya Test1 : 78 Test2 : 82 Improvement =  4
Names : Ishaan Test1 : 45 Test2 : 50 Improvement =  5
Names : Meera Test1 : 88 Test2 : 90 Improvement =  2
Names : Kabir Test1 : 62 Test2 : 58 Improvement =  -4
Names : Anaya Test1 : 91 Test2 : 94 Improvement =  3
Names : Rohan Test1 : 40 Test2 : 38 Improvement =  -2
Names : Sara Test1 : 73 Test2 : 70 Improvement =  -3
Names : Vivaan Test1 : 67 Test2 : 72 Improvement =  5
Names : Priya Test1 : 85 Test2 : 88 Improvement =  3
Names : Arjun Test1 : 52 Test2 : 47 Improvement =  -5
```

Spaces: 4 CRLF Cell 4 of 30

A4. Condionals :

```
count =0
for i in range(len(names)):
    if(test1[i] > 80):
        print(names[i])
        count+=1
print(count)
```

[9] ✓ 0.0s Python

```
Meera
Anaya
Priya
Nisha
4
```

A5. Functions :

```
def average(values):
    total =0
    for value in values:
        total+=value
    avg=total/len(values)

    return avg

print("Average of Test 1 : ", average(test1))
print("Average of Test 2 : ", average(test2))
print("Average of Attendance : ", average(attendance))
```

[10] ✓ 0.0s Python

```
Average of Test 1 :  69.33333333333333
Average of Test 2 :  70.58333333333333
Average of Attendance :  79.33333333333333
```

The average of Test2 is higher than Test1. This means the class performed better in Test2 overall.

No. Average alone cannot tell whether every student improved because some students may improve while others may score lower.

Part B : Data as Vectors with Numpy

B1. From lists to vectors :

```
import numpy as np
t1= np.array(test1)
t2= np.array(test2)
t3=np.array(attendance)
print(t2-t1)
```

[11] ✓ 0.2s Python

```
[ 5  4  5  2 -4  3 -2 -3  5  3 -5  2]
```

B2. Summary Stats :

```
print("Mean : " , np.mean(t1))
print("Variance : ", np.var(t1))
print("STD Deviation : ", np.std(t1))
```

[12] ✓ 0.0s Python

```
Mean :  69.33333333333333
Variance :  316.72222222222223
STD Deviation :  17.796691327946952
```

B3. Interpreting spread :

```
print("test1 : " ,np.std(t1))
print("Atte : ",np.std(t3))
```

[13] ✓ 0.0s Python

```
test1 :  17.796691327946952
Atte :  14.522013940527977
```

Test1 marks are more spread out than attendance. A larger standard deviation means the data values are more dispersed from the average.

B4. A first taste of geometry :

```
meera  = np.array([88, 90, 95])
kabir  = np.array([62, 58, 70])
dist = np.sqrt(np.sum((meera - kabir)**2))
print(dist)
```

[14] ✓ 0.0s Python

```
48.218253804964775
```

```
students = np.column_stack((t1, t2, t3))

nisha = students[-1]

(variable) closest_student: Literal['']
closest_student = ""

for i in range(len(names) - 1):
    distance = np.sqrt(np.sum((students[i] - nisha) ** 2))

    if distance < min_distance:
        min_distance = distance
        closest_student = names[i]

print("Student closest to Nisha:", closest_student)
print("Distance:", min_distance)
```

[15] ✓ 0.0s Python

```
Student closest to Nisha: Anaya
Distance: 5.0990195135927845
```

Part C : First look at real EDA with pandas

C1. Load and inspect :

```
import pandas as pd
df = pd.DataFrame({ "name" : names , "test1" : test1 , "test2" : test2 , "attendance" : attendance} )
df.to_csv("marks.csv", index=False)
```

[16] ✓ 0.6s Python

```
pd.read_csv("marks.csv")
df.head()
df.shape
df.describe()
```

[17] ✓ 0.0s Python

	test1	test2	attendance
count	12.000000	12.000000	12.000000
mean	69.333333	70.583333	79.333333
std	18.588038	19.888134	15.167749
min	40.000000	38.000000	55.000000
25%	55.000000	56.000000	68.750000
50%	70.000000	71.000000	79.000000
75%	85.750000	88.500000	92.750000
max	95.000000	97.000000	99.000000

C2. Selecting and Filtering :

```
print(df[["name" , "test1"]])
```

[18] ✓ 0.0s Python

```
      name  test1
0    Aarav     56
1     Diya     78
2   Ishaan     45
3    Meera     88
4    Kabir     62
5    Anaya     91
6    Rohan     40
7     Sara     73
8   Vivaan     67
9    Priya     85
10   Arjun     52
11   Nisha     95
```

```
print(df[df["attendance"] > 85])
```

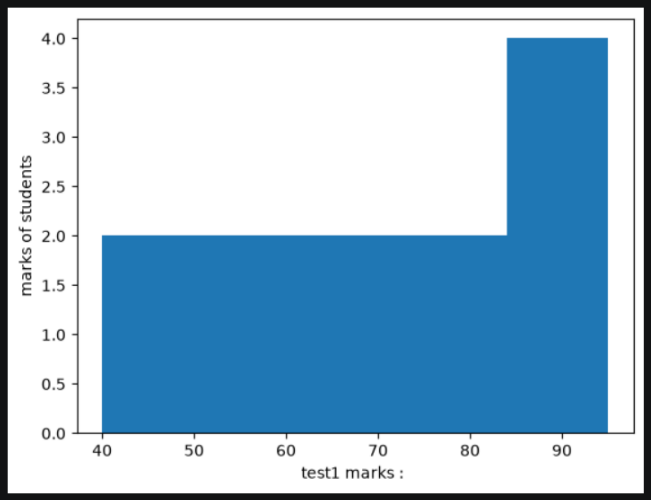
[25] ✓ 0.0s Python

```
      name  test1  test2  attendance grade
1     Diya     78     82          88     B
3    Meera     88     90          95     A
5    Anaya     91     94          98     A
9    Priya     85     88          92     A
11   Nisha     95     97          99     A
```

C3. Histogram and your first distribution :

```
import matplotlib.pyplot as plt
plt.hist(df["test1"] , bins=5)
plt.xlabel("test1 marks : ")
plt.ylabel("marks of students")
plt.show()
```

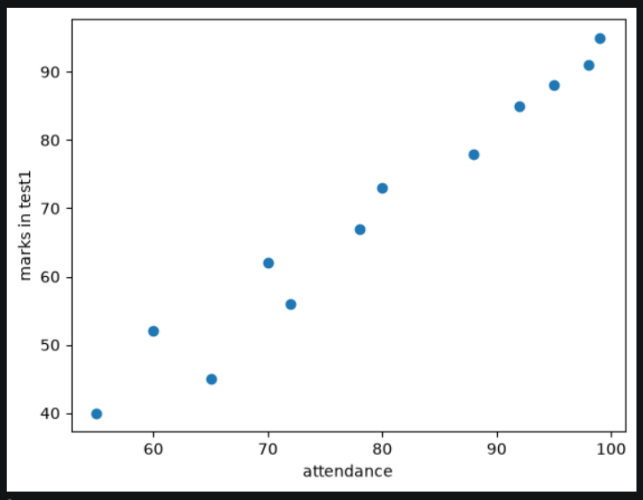
[20] ✓ 1.0s Python



C4. Scatter Plot :

```
plt.scatter(df["attendance"] , df["test1"])
plt.xlabel("attendance")
plt.ylabel("marks in test1")
plt.show()
```

[21] ✓ 0.1s Python



Home Challenge :

D1: Write a function grade(marks) that returns "A" for 85 and above, "B" for 70–84, "C" for 50–69, and "F" below 50. Add a new column "grade" to the DataFrame for Test 2 and print how many students got each grade (hint: df["grade"].value_counts()).

```
def grade(marks):
    if marks>=85:
        return "A"
    if marks >= 70:
        return "B"
    if marks >= 50:
        return "C"
    else :
        return "F"

df["grade"]= df["test2"].apply(grade)
print(df)
df["grade"].value_counts()
```

[22] ✓ 0.0s Python

```
      name  test1  test2  attendance grade
0    Aarav     56     61          72     C
1     Diya     78     82          88     B
2   Ishaan     45     50          65     C
3    Meera     88     90          95     A
4    Kabir     62     58          70     C
5    Anaya     91     94          98     A
6    Rohan     40     38          55     F
7     Sara     73     70          80     B
8   Vivaan     67     72          78     B
9    Priya     85     88          92     A
10   Arjun     52     47          60     F
11   Nisha     95     97          99     A

grade
A    4
C    3
B    3
F    2
Name: count, dtype: int64
```

CRLF Cell 21 of 30

D2. One student's Test 1 entry was mistyped: change Rohan's test1 to 400 and recompute the mean and standard deviation. Write 2–3 sentences on how a single wrong value distorted both numbers.

```
df.loc[df["name"] == "Rohan" , "test1"] = 400
print("updated mean : ",df["test1"].mean)
print("updated STD : ", df["test1"].std)
```

[23] ✓ 0.0s Python

```
updated mean :  <bound method Series.mean of 0      56
1      78
2      45
3      88
4      62
5      91
6     400
7      73
8      67
9      85
10     52
11     95
Name: test1, dtype: int64>
updated STD :  <bound method Series.std of 0      56
1      78
2      45
3      88
4      62
5      91
6     400
7      73
8      67
9      85
10     52
11     95
Name: test1, dtype: int64>
```

Changing Rohan's marks from 40 to 400 created a change in data. The mean increased very high because it large values. The standard deviation also increased because the data became much more spread out. This shows that a single incorrect value can distort statistical measures.

D3. Optional stretch: without using np.var, compute the variance of test1 yourself using only a loop and the formula: average of squared differences from the mean. Verify it matches np.var(t1).

```
values = test1

mean = sum(values) / len(values)

total = 0

for value in values:
    total += (value - mean) ** 2

variance = total / len(values)

print("Manual Variance:", variance)
print("Automated var :",np.var(t1))
```

[24] ✓ 0.0s Python

```
Manual Variance: 316.7222222222222
Automated var : 316.72222222222223
```